

MODULE 07 · 25 MIN

Safer & smarter

Claude Code 101 · Beginner Workshop · Module 7 of 8

The model is helpful, polite, and remembers what you paste. Three things you should always think twice before sending.

What you'll learn

By the end of this 25-minute lesson you will be able to:

- 1 List the three categories never to paste into Claude: secrets, PII, and proprietary data you don't own.
- 2 Recognise them in a real file before you copy.
- 3 Choose the right substitute: a fake value, a redacted snippet, or a "don't share" decision.

Why this matters

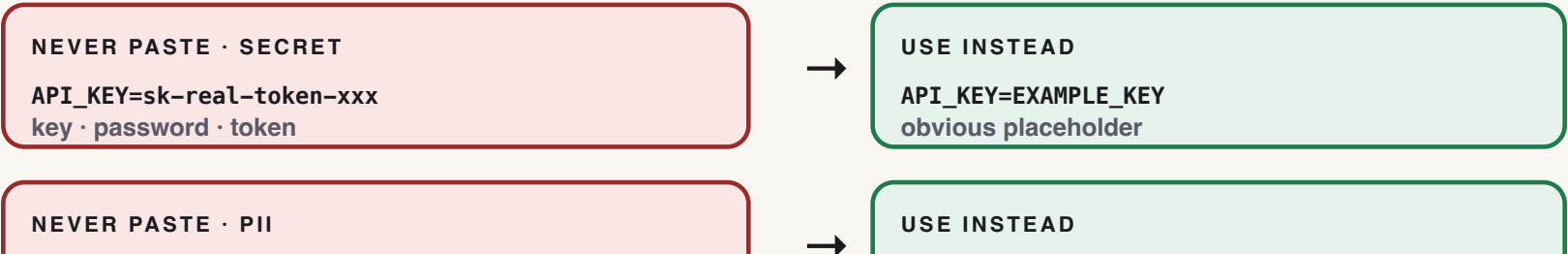
- Anything you paste leaves your machine. Even if the provider promises not to train on it, "we promise" is not a security control.
- Secrets in chat logs end up in screenshots, in backups, in support tickets. The blast radius is much larger than the chat window.
- Most accidents are not malicious — they happen when someone copy-pastes a config file at 4:55pm. A 10-second redaction habit prevents 100% of those accidents.

The one concept

Three categories. Three reactions. No exceptions.

Category	Examples	What to do instead
Secret	API key, password, token, private SSH key, database URL with creds	Replace with <code>EXAMPLE_KEY</code> or <code>xxx</code> . Then paste.
PII	Real name + email, phone, address, customer record, internal user list	Replace with <code>Alice / alice@example.com</code> . Then paste.
Proprietary / under NDA	Customer source code you don't own, internal designs, unannounced product	Don't paste. Describe the shape ("a 50-line auth handler in Go").

If you cannot honestly redact something, you cannot paste it. That is the test.



Show me

Wrong — a real connection string with a real password:

```
> Why does this connection fail?  
DATABASE_URL=postgres://admin:Hunter2-real-password@prod-db.acme.com:5432/customers
```

Right — same shape, no actual secrets:

```
> Why does this connection fail?  
DATABASE_URL=postgres://USER:PASSWORD@HOST:5432/DBNAME
```

Claude can debug the second one just as well as the first. The first one might end up in a support transcript, a screenshot, or a colleague's chat history.

► Try it yourself

`exercises/beginner/part-07/starter/leaky_config.txt` contains a fake config file with three planted leaks. Your job: redact all three and save the cleaned version. The exercise asks Claude **only** about the redacted version.

Time budget: 10 minutes. None of the leaks are real — they are obvious placeholders so nobody can be hurt if you slip.

Common mistakes

- **"It's just localhost"** — `localhost` URLs can still contain real tokens in query strings.
- **"It's just a test account"** — test accounts share infrastructure with prod more often than you think.
- **"The model promised not to train on it"** — irrelevant. The risk is logs, screenshots, and pastes-of-the-paste.
- **Redacting only the obvious one** — config files leak in surprising places (comments, env vars, file paths).
- **Asking Claude to redact it for you, then sending the original to be redacted.** That's the leak.

✓ Lesson reflection

Take 90 seconds:

1. In the last week, did you paste anything into a chatbot that fit one of the three categories?
2. Look at your last terminal scrollback. Is there anything you'd be uncomfortable sharing on Slack? That's the same standard.
3. If a colleague asked you "is it safe to paste this?", what one-sentence rule would you give them?

What's next →

Module 08 — **Putting it together** — is the capstone. You'll build a tiny Python CLI using everything from Modules 01–07, and a script will grade your result.

Budget for Module 08: 30 minutes.

Glossary card

- **Permission:** A yes/no setting that lets a tool read, write, or run things on your machine.
- **PII:** Personally identifiable information — names, addresses, phone numbers, account numbers, and similar private data.
- **Secret:** Sensitive data like a password or API key that must never be pasted into a prompt.